

NumPy Tricks & Hidden Functions

Data Science Mentorship Program | Verified Code & Outputs

1. Sorting Arrays (`np.sort`)

Instead of Python's built-in `sorted()` which returns a list, `np.sort()` returns a sorted NumPy array.

```
import numpy as np

arr = np.array([45, 12, 89, 33, 2])
print(np.sort(arr))
```

OUTPUT

```
[ 2 12 33 45 89]
```

2. Modifying Array Content

Appending (`np.append`)

Appends items to the end of an array. Note: For 2D arrays, specify the axis (0 for rows, 1 for columns).

```
arr = np.array([1, 2, 3])
print(np.append(arr, 200))
```

OUTPUT

```
[ 1  2  3 200]
```

Concatenating (`np.concatenate`)

Used heavily as an alternative to `vstack` and `hstack`.

```
c = np.array([[1, 2], [3, 4]])
d = np.array([[5, 6], [7, 8]])

# Axis 0 (Row-wise / Vertical)
print(np.concatenate((c, d), axis=0))
```

OUTPUT

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

3. Finding Unique Values (`np.unique`)

Returns only the unique elements from an array. Heavily used in data analysis.

```
arr = np.array([1, 1, 2, 2, 3, 4, 4, 4])
print(np.unique(arr))
```

OUTPUT

```
[1 2 3 4]
```

4. Expanding Dimensions (`np.expand_dims`)

Adds an extra dimension to the array. Crucial for Deep Learning/ML when models expect batches of data rather than single inputs.

```
arr = np.array([1, 2, 3]) # Shape: (3,)
expanded_arr = np.expand_dims(arr, axis=0)
print(expanded_arr)
print("Shape:", expanded_arr.shape)
```

OUTPUT

```
[[1 2 3]]
Shape: (1, 3)
```

5. Conditional Logic & Filtering (`np.where`)

Extremely powerful. It can return indices where a condition is met, OR conditionally replace values.

```
arr = np.array([10, 60, 20, 80, 30])

# Get index positions where condition is True
print(np.where(arr > 50))
```

OUTPUT

```
(array([1, 3]),)
```

```
# Replace values: np.where(condition, if_true, if_false)
# E.g., Replace numbers > 50 with 0
print(np.where(arr > 50, 0, arr))
```

OUTPUT

```
[10  0 20  0 30]
```

6. Finding Max/Min Indices (`np.argmax` & `np.argmin`)

Instead of the actual max value, these return the **index position** of the maximum/minimum value.

```
arr = np.array([10, 60, 99, 80, 2])
print("Max Index:", np.argmax(arr))
print("Min Index:", np.argmin(arr))
```

OUTPUT

```
Max Index: 2
Min Index: 4
```

7. Statistical Tricks

Cumulative Sum (`np.cumsum`)

Calculates the running total (e.g., career total runs match by match).

```
arr = np.array([25, 18, 59, 100])
print(np.cumsum(arr))
```

OUTPUT

```
[ 25  43 102 202]
```

Percentiles (`np.percentile`)

Finds the value below which a given percentage of observations fall.

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
print("50th Percentile (Median):", np.percentile(arr, 50))
```

OUTPUT

```
50th Percentile (Median): 55.0
```

8. Set Operations & Search

Multiple Item Search (`np.isin`)

Checks if elements of the array are present in a target list/array. Great for filtering.

```
arr = np.array([10, 20, 30, 40, 50])
search_items = [20, 50, 100]

print(np.isin(arr, search_items))
```

OUTPUT

```
[False  True False False  True]
```

Set Functions

Perform standard set operations returning NumPy arrays.

```
arr1 = np.array([1, 2, 3, 4, 5])
arr2 = np.array([3, 4, 5, 6, 7])

print("Union:", np.union1d(arr1, arr2))
print("Intersection:", np.intersect1d(arr1, arr2))
print("Set Difference:", np.setdiff1d(arr1, arr2))
```

OUTPUT

```
Union: [1 2 3 4 5 6 7]
Intersection: [3 4 5]
Set Difference: [1 2]
```

9. Array Boundaries (`np.clip`)

Keeps array values strictly within a specified minimum and maximum range. Values outside are replaced by the boundaries.

```
arr = np.array([10, 50, 90, 120, 5])  
# Keep values between 25 and 75  
print(np.clip(arr, a_min=25, a_max=75))
```

OUTPUT

```
[25 50 75 75 25]
```